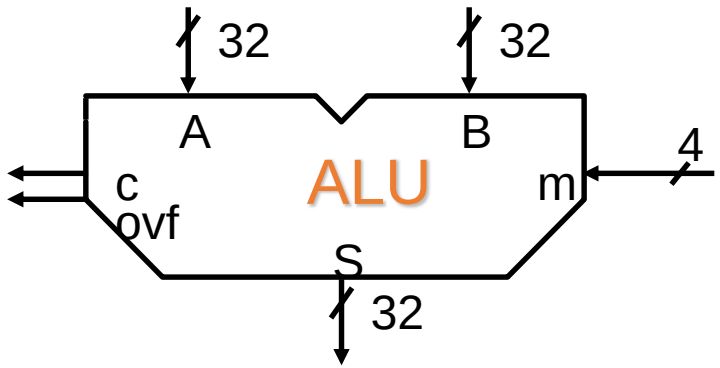
Computer Arithmetic



Digital Systems

LNMIIT
The LNM Institute of Information Technology

Representation of Signed Integers

LNMIIT
The LNM Institute of Information Technology

- Sign-and-magnitude
- Two's-complement
- Ones'-complement

Representation of Signed Integers

Decimal	Signed 2's Complement	Signed 1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
−0	—	1111	1000
−1	1111	1110	1001
−2	1110	1101	1010
−3	1101	1100	1011
−4	1100	1011	1100
−5	1011	1010	1101
−6	1010	1001	1110
−7	1001	1000	1111
−8	1000	—	—

Benefits of Two's Compliment

- One representation of zero
- Arithmetic works easily (see later)
- Negating is fairly easy
  - 3 = 00000011
  - Boolean complement gives 11111100
  - Add 1 to LSB 11111101

Range of Numbers

- 8 bit 2s compliment
  - +127 = 01111111 = 2^7 -1
  - -128 = 10000000 = -2^7
- 16 bit 2s compliment
  - +32767 = 011111111 11111111 = 2^{15} - 1
  - -32768 = 100000000 00000000 = -2^{15}

Conversion Between Lengths

LNMIIT
The LNM Institute of Information Technology

- Positive number pack with leading zeros
- +18 = 00010010
- +18 = 00000000 00010010
- Negative numbers pack with leading ones
- -18 = 10010010
- -18 = 11111111 10010010
- i.e. pack with MSB (sign bit)

Addition and Subtraction

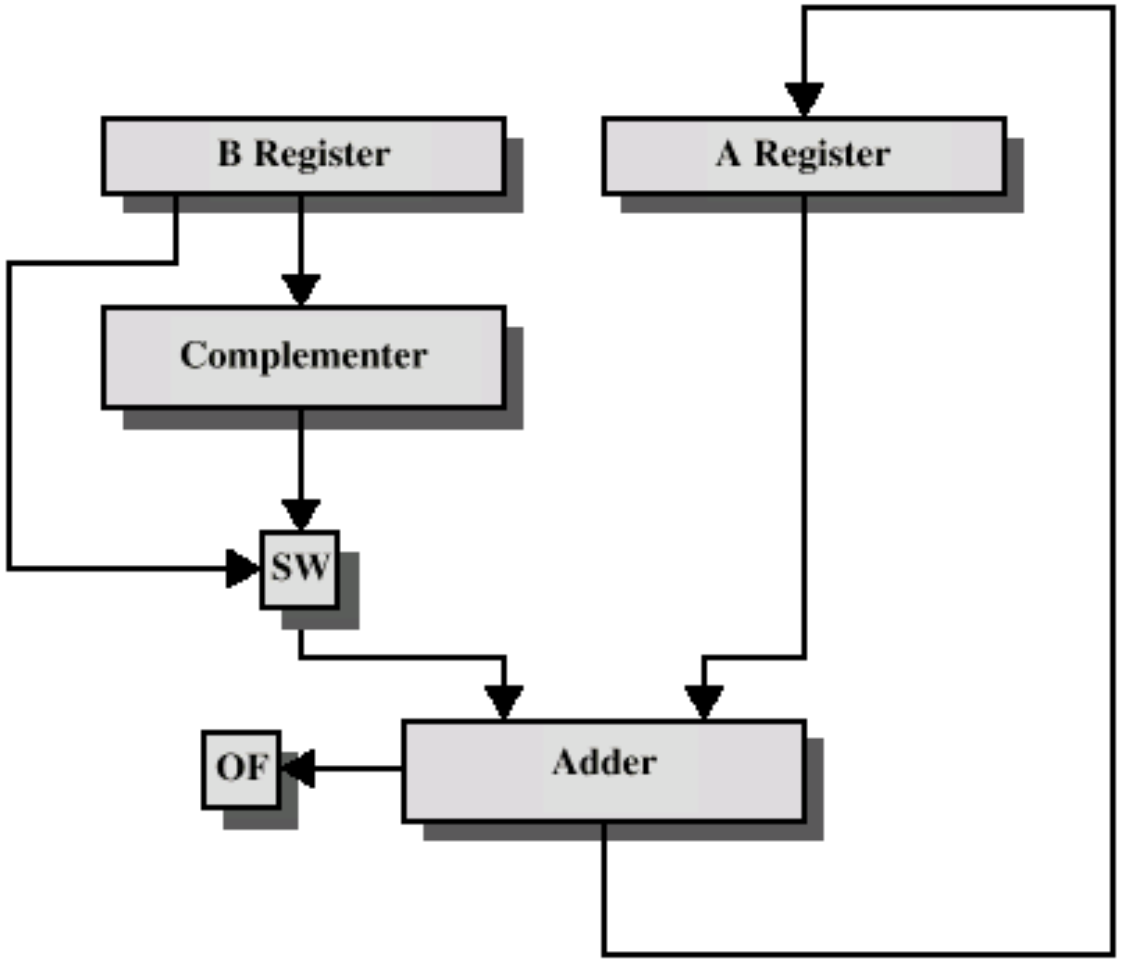
LNMIIT
The LNM Institute of Information Technology

- Normal binary addition
- Monitor sign bit for overflow

- Take twos compliment of substahend and add to minuend
  - i.e. a - b = a + (-b)

- So we only need addition and complement circuits

Hardware for Addition and Subtraction



OF = overflow bit
SW = Switch (select addition or subtraction)

Multiplication

- Complex
- Work out partial product for each digit
- Take care with place value (column)
- Add partial products

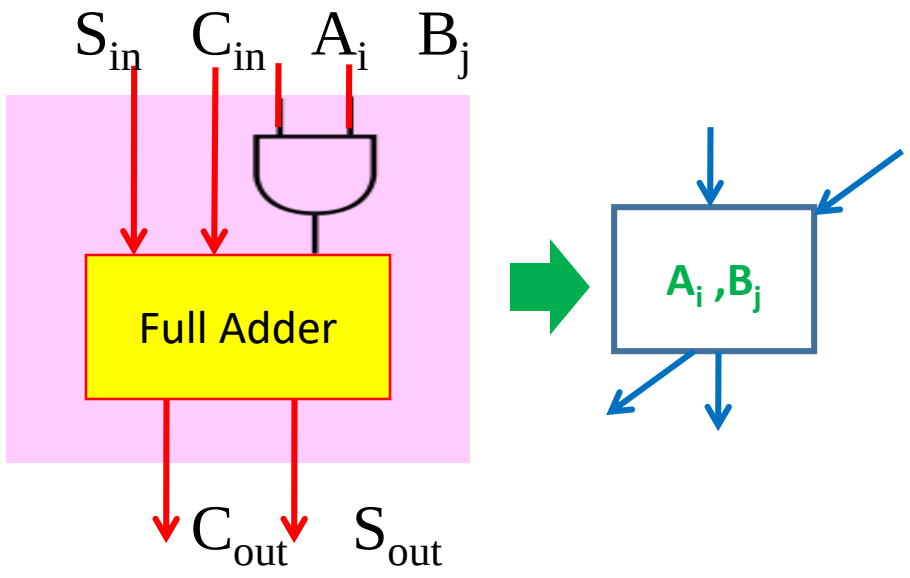
Multiplication Example

- 1011 Multiplicand (11 dec)
- x 1101 Multiplier (13 dec)
- 1011 Partial products
- 0000 Note: if multiplier bit is 1 copy
- 1011 multiplicand (place value)
- 1011 otherwise zero
- 10001111 Product (143 dec)
- Note: need double length result

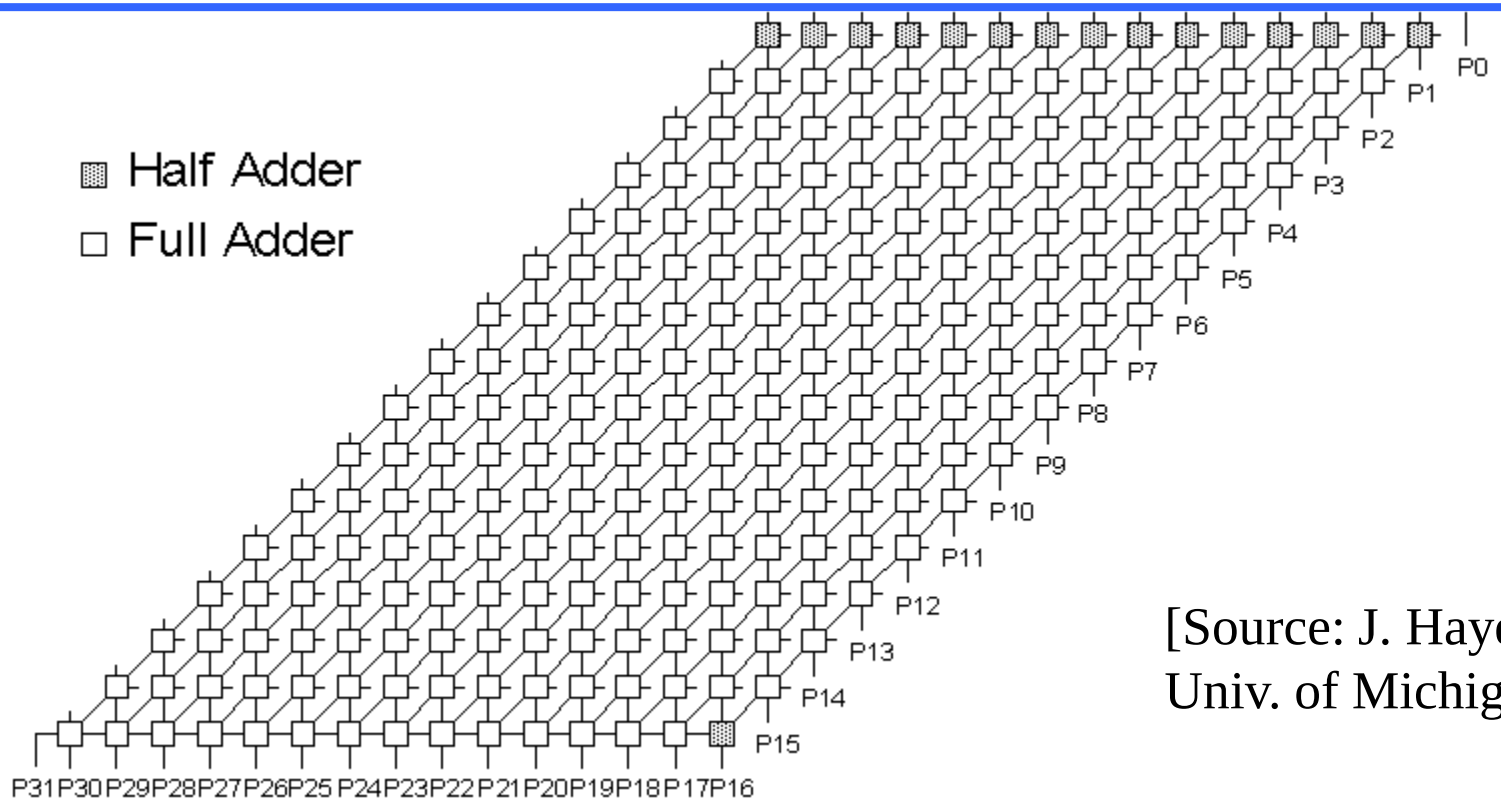
Array Multiplier

			1	0	0	0
		x	1	0	0	1
			1	0	0	0
		0	0	0	0	
	0	0	0	0		
1	0	0	0			
1	0	0	1	0	0	0

- Adding all partial products simultaneously using an array of basic cells



16-bit Array Multiplier



[Source: J. Hayes, Univ. of Michigan]

Conceptually straightforward

Fairly expensive hardware, integer multiplies relatively rare

Most used in array address calc: replace with shifts

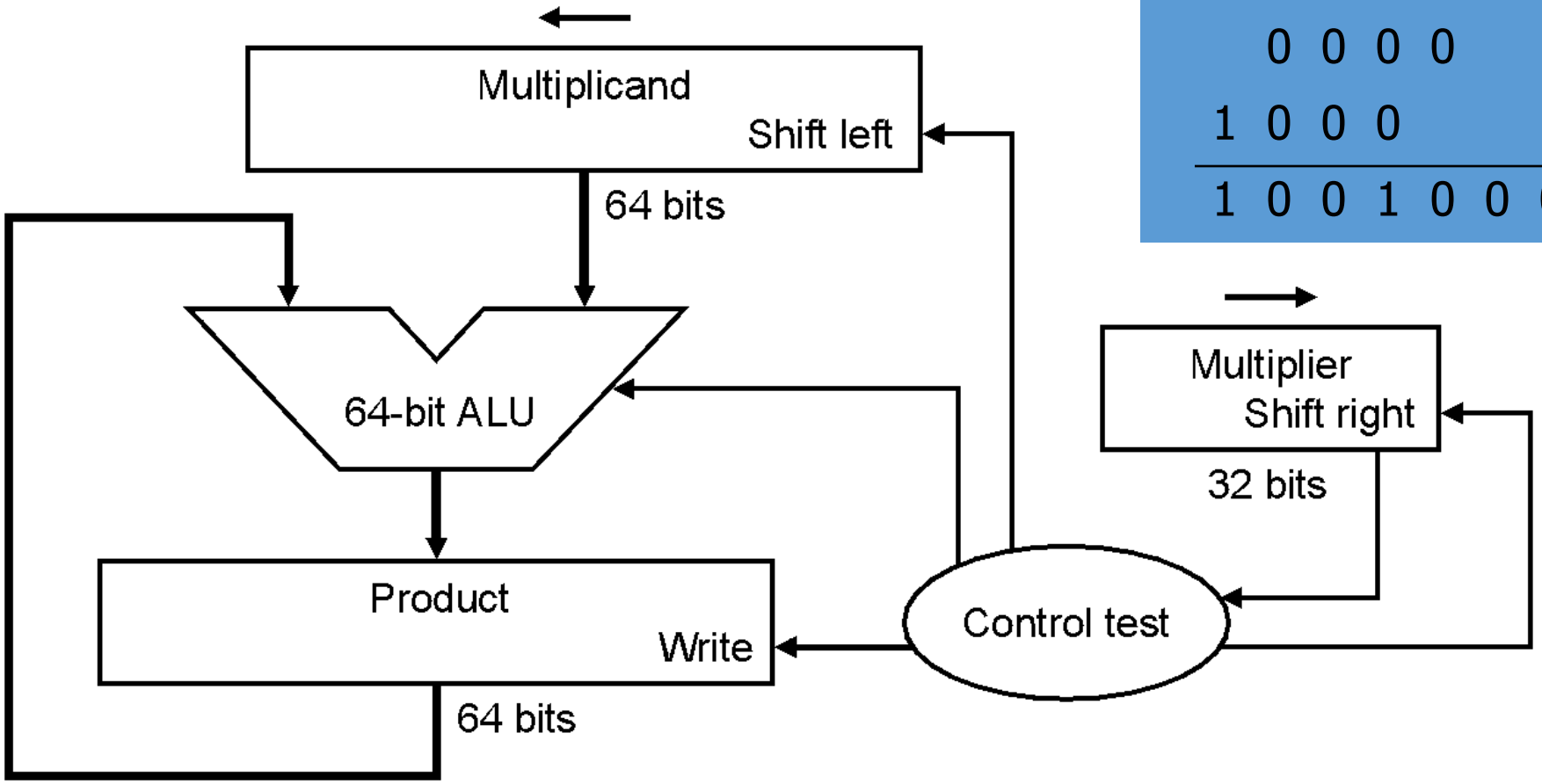
Instead: Multicycle Multipliers

LNMIIT
The LNM Institute of Information Technology

- Combinational multipliers
  - Very hardware-intensive
  - Integer multiply relatively rare
  - Not the right place to spend resources
- Multicycle multipliers
  - Iterate through bits of multiplier
  - Conditionally add shifted multiplicand

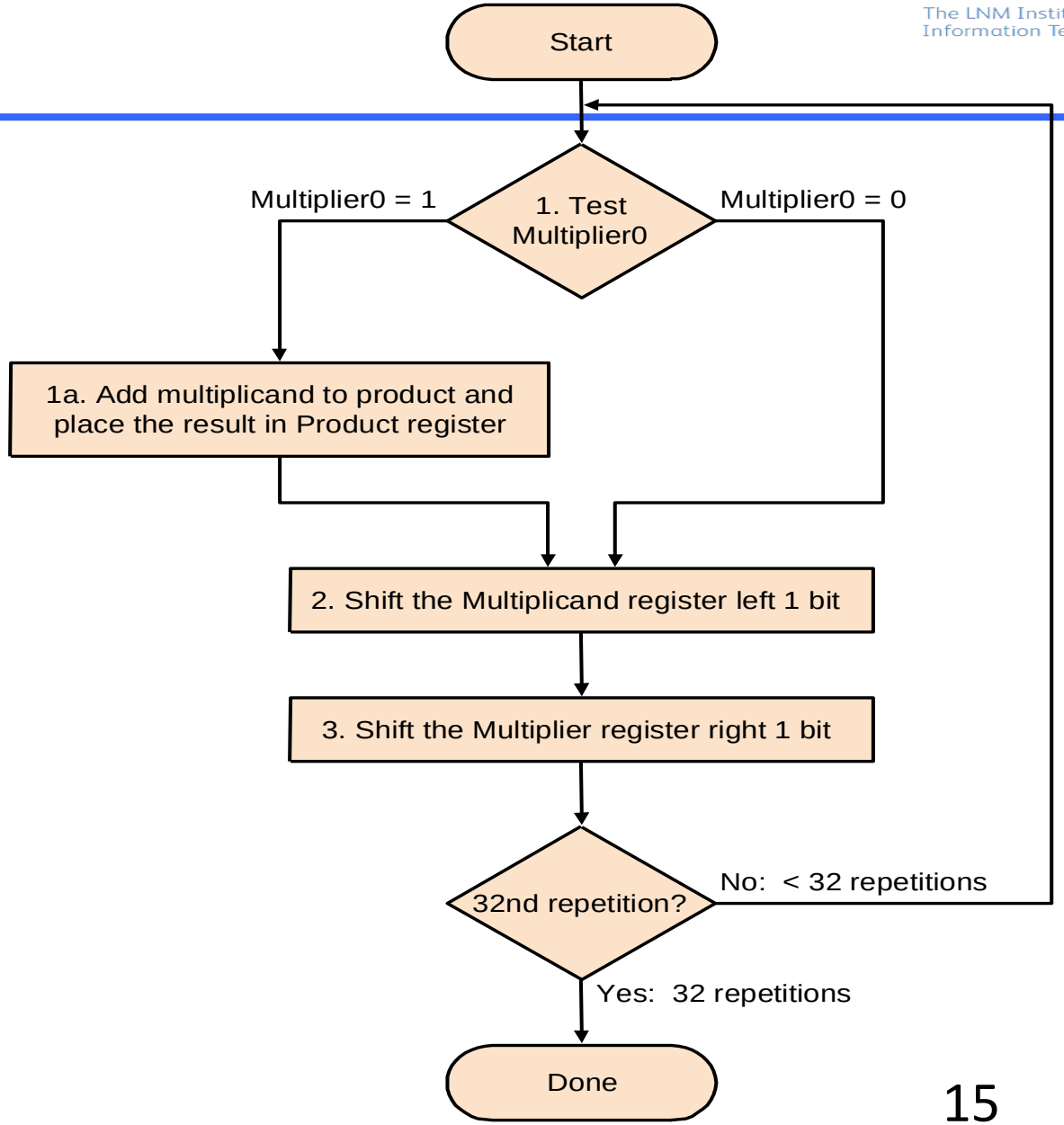
Multiplier

				1	0	0	0
			x	1	0	0	1
				1	0	0	0
		0	0	0	0		
	0	0	0	0			
1	0	0	0				
1	0	0	1	0	0	0	

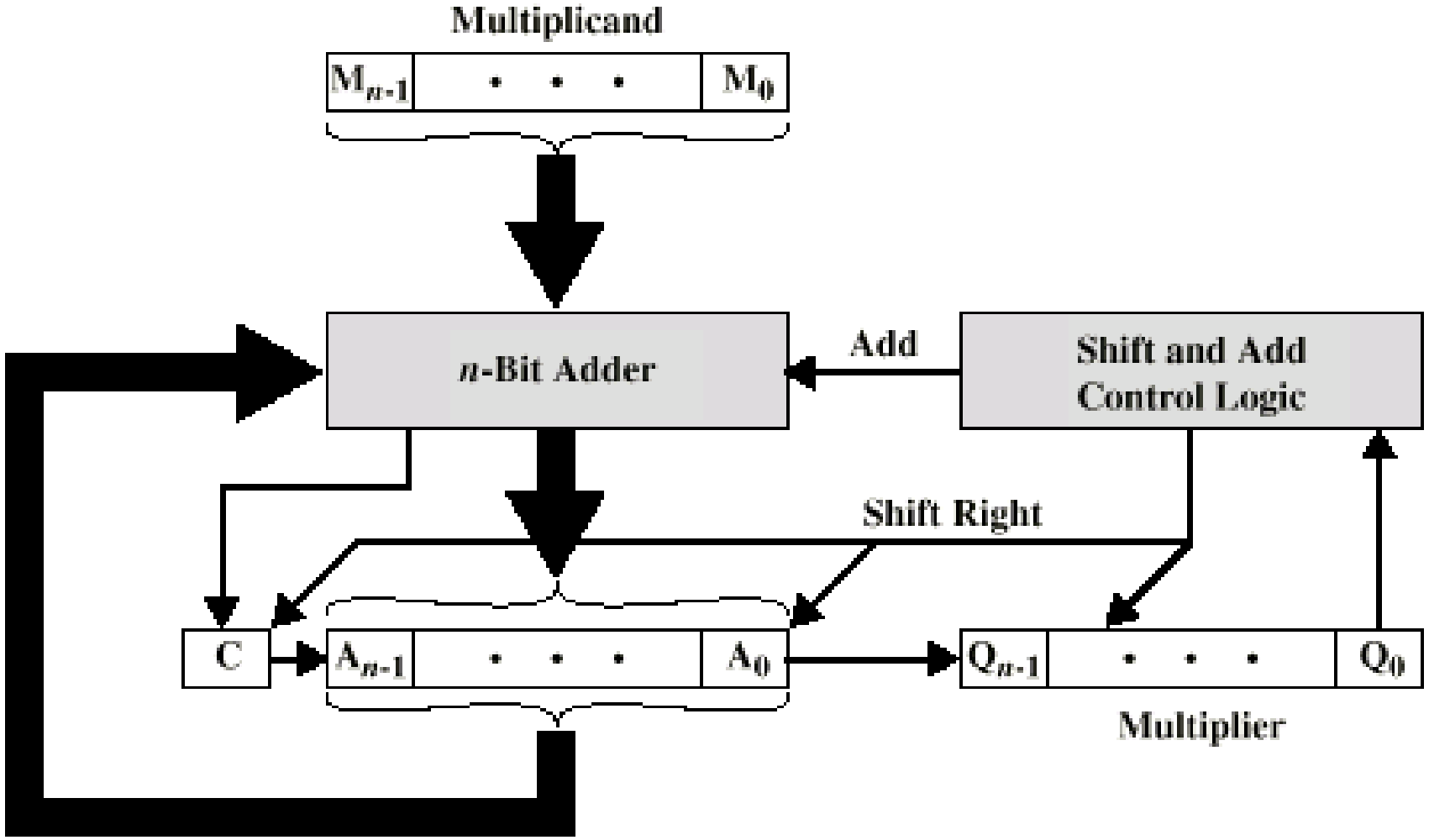


Multiplier

				1	0	0	0
			x	1	0	0	1
				1	0	0	0
			0	0	0	0	
		0	0	0	0		
	1	0	0	0			
1	0	0	1	0	0	0	



Unsigned Binary Multiplication

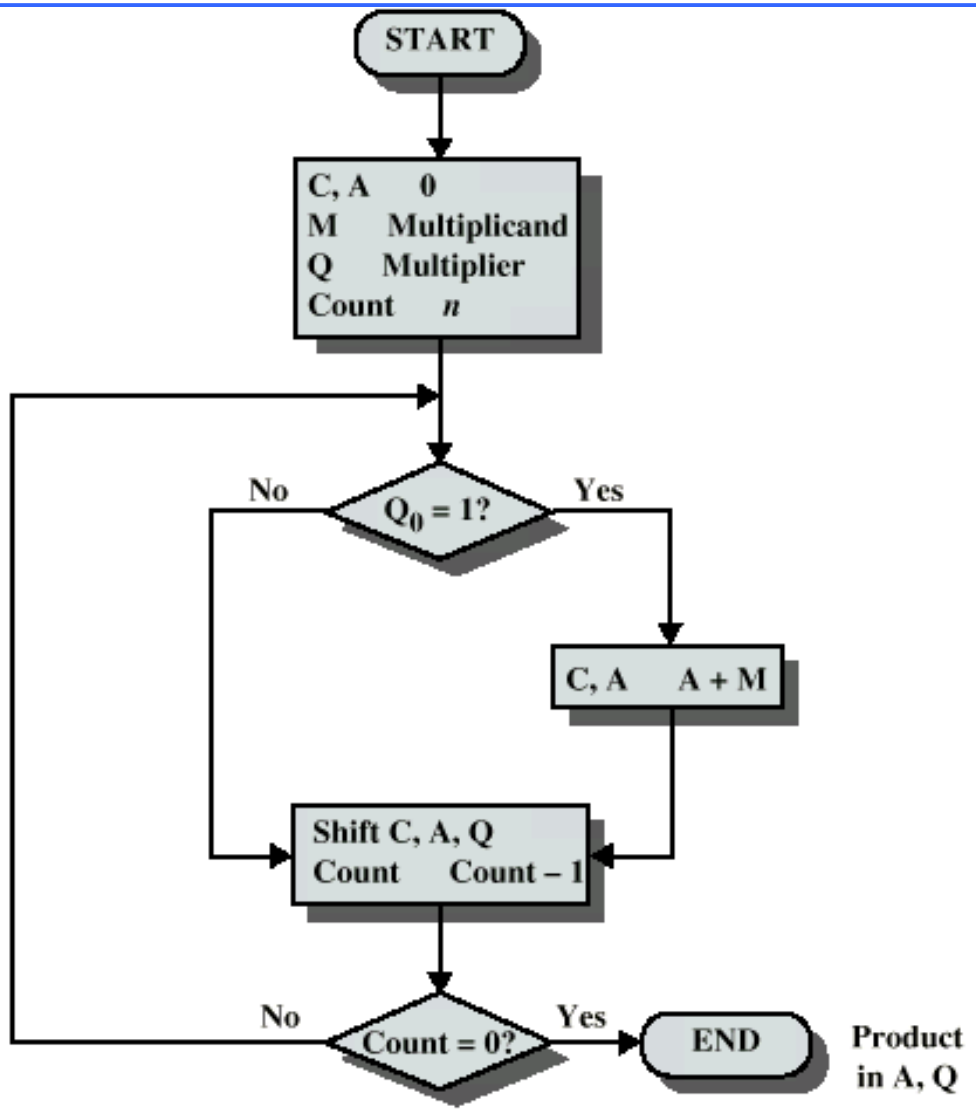


(a) Block Diagram

Execution of Example

C	A	Q	M		
0	0000	1101	1011	Initial Values	
0	1011	1101	1011	Add	First Cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	Shift	Second Cycle
0	1101	1111	1011	Add	Third Cycle
0	0110	1111	1011	Shift	
1	0001	1111	1011	Add	Fourth Cycle
0	1000	1111	1011	Shift	

Flowchart for Unsigned Binary Multiplication

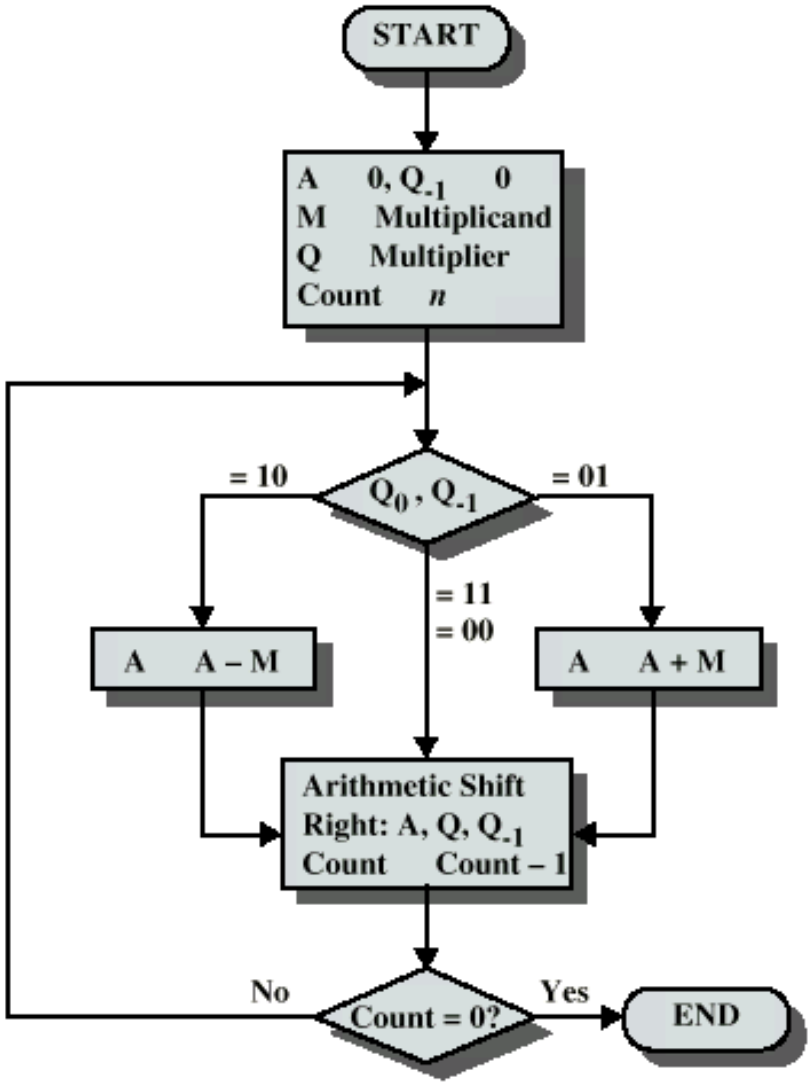


Multiplying Negative Numbers

LNMIIT
The LNM Institute of Information Technology

- This does not work!
- Solution 1
  - Convert to positive if required
  - Multiply as above
  - If signs were different, negate answer
- Solution 2
  - Booth's algorithm

Booth's Algorithm



Booth's Encoding

LNMIIT
The LNM Institute of Information Technology

- Search for a run of '1' bits in the multiplier
  - E.g. '0110' has a run of 2 '1' bits in the middle
  - Multiplying by '0110' (6 in decimal) is equivalent to multiplying by 8 and subtracting twice, since 6 x m = (8 – 2) x m = 8m – 2m
- Hence, iterate right to left and:
  - Subtract multiplicand from product at first '1'
  - Add multiplicand to product after last '1'
  - Don't do either for '1' bits in the middle

Booth's Algorithm

LNMIIT
The LNM Institute of Information Technology

Current bit	Bit to right	Explanation	Example	Operation
1	0	Begins run of '1'	00001111000	Subtract
1	1	Middle of run of '1'	00001111000	Nothing
0	1	End of a run of '1'	00001111000	Add
0	0	Middle of a run of '0'	00001111000	Nothing

Booth's Encoding

- Really just a new way to encode numbers
  - Normally positionally weighted as 2^n
  - With Booth, each position has a sign bit
  - Can be extended to multiple bits

0	1	1	0	Binary
+1	0	-1	0	1-bit Booth
+2		-2		2-bit Booth

2-bits/cycle Booth Multiplier

LNMIIT
The LNM Institute of Information Technology

- For every pair of multiplier bits
  - If Booth's encoding is '-2'
    - Shift multiplicand left by 1, then subtract
  - If Booth's encoding is '-1'
    - Subtract
  - If Booth's encoding is '0'
    - Do nothing
  - If Booth's encoding is '1'
    - Add
  - If Booth's encoding is '2'
    - Shift multiplicand left by 1, then add

2 bits/cycle Booth's

1 bit Booth	
00	+0
01	+M;
10	-M;
11	+0

Current	Previous	Operation	Explanation
00	0	+0;shift 2	[00] => +0, [00] => +0; 2x(+0)+(+0)=+0
00	1	+M; shift 2	[00] => +0, [01] => +M; 2x(+0)+(+M)=+M
01	0	+M; shift 2	[01] => +M, [10] => -M; 2x(+M)+(-M)=+M
01	1	+2M; shift 2	[01] => +M, [11] => +0; 2x(+M)+(+0)=+2M
10	0	-2M; shift 2	[10] => -M, [00] => +0; 2x(-M)+(+0)=-2M
10	1	-M; shift 2	[10] => -M, [01] => +M; 2x(-M)+(+M)=-M
11	0	-M; shift 2	[11] => +0, [10] => -M; 2x(+0)+(-M)=-M
11	1	+0; shift 2	[11] => +0, [11] => +0; 2x(+0)+(+0)=+0

Booth's Example

- Negative multiplicand:

  -6 x 6 = -36

  1010 x 0110, 0110 in Booth's encoding is +0-0

  Hence:

1111 1010	x 0	0000 0000
1111 0100	x –1	0000 1100
1110 1000	x 0	0000 0000
1101 0000	x +1	1101 0000
	Final Sum:	1101 1100 (-36)

Booth's Example

- Negative multiplier:

  -6 x -2 = 12

  1010 x 1110, 1110 in Booth's encoding is 00-0

  Hence:

1111 1010	x 0	0000 0000
1111 0100	x –1	0000 1100
1110 1000	x 0	0000 0000
1101 0000	x 0	0000 0000
	Final Sum:	0000 1100 (12)

Example of Booth's Algorithm

Find 3 × (−4), with **m** = 3 and **r** = −4:

- m = 0011, -m = 1101, r = 1100
- A = 0011 0000 0
- S = 1101 0000 0
- P = 0000 1100 0
- Perform the loop four times:
  - P = 0000 110**0 0**. The last two bits are 00.
    - P = 0000 0110 0. Arithmetic right shift.
  - P = 0000 011**0 0**. The last two bits are 00.
    - P = 0000 0011 0. Arithmetic right shift.
  - P = 0000 001**1 0**. The last two bits are 10.
    - P = 1101 0011 0. P = P + S.
    - P = 1110 1001 1. Arithmetic right shift.
  - P = 1110 100**1 1**. The last two bits are 11.
    - P = 1111 0100 1. Arithmetic right shift.
- The product is 1111 0100, which is −12.

Division

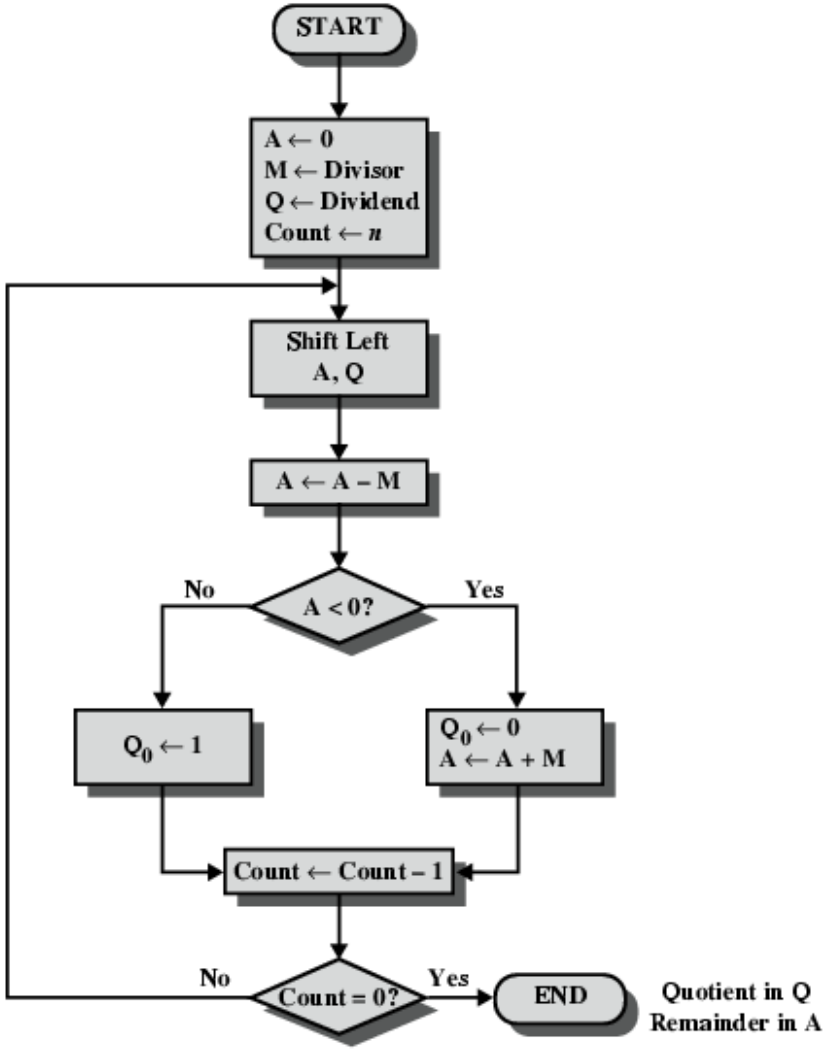
- More complex than multiplication
- Negative numbers are really bad!
- Based on long division

Division of Unsigned Binary Integers

```
                 00001101   ← Quotient
          ┌──────────────
Divisor → 1011 │ 10010011   ← Dividend
                 1011
                 ────
                 001110
                   1011
                   ────
                   001111
                     1011
                     ────
                      100   ← Remainder
```

Partial Remainders

Flowchart for Unsigned Binary Division



Real Numbers

- Numbers with fractions
- Could be done in pure binary
  - $1001.1010 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 9.625$
- Where is the binary point?
- Fixed?
  - Very limited
- Moving?
  - How do you show where it is?

Floating Point Arithmetic

- How to represent
  - numbers with fractions, e.g., 3.1416
  - very small numbers, e.g., .000000001
  - very large numbers, e.g., 3.15576×10^9
- Fixed point
- Floating point: a number system with floating decimal point
- Normalized numbers: no leading 0's , single digit before decimal point

1.0 x 10^{-9}
3.1557 x 10^{9}
35
0.03

IEEE 754 – Floating point Representation

LNMIIT
The LNM Institute of Information Technology

- The standard defined by IEEE std 754-1985
- Now almost universally adopted
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)

Floating Point standard

- Single Precision

s	e (8-bits)	f (23-bits)

- Value of bits stored in representation is:
  - If e=255 and f /= 0, then v is NaN regardless of s
    - Not-a-Number (NaN): Indicates illegal or undefined result e.g., 0.0 / 0.0
  - If e=255 and f = 0, then v = $(-1)^s \infty$
  - If $0 < e < 255$, then v = $(-1)^s 2^{e-127} (1.f)$ – normalized number
  - If e = 0 and f /= 0, the v = $(-1)^s 2^{-126} (0.f)$
    - Denormalized numbers – allow for graceful underflow
  - If e = 0 and f = 0 the v = $(-1)^s 0$ (zero)

Floating Point standard

- Double Precision

s	e (11-bits)	f (52-bits)

- Value of bits in word representation is:
  - If e=2047 and f /= 0, then v is NaN regardless of s
  - If e=2047 and f = 0, then v = $(-1)^s \infty$
  - If $0 < e < 2047$, then v = $(-1)^s 2^{e-1023} (1.f)$
    - – normalized number
  - If e = 0 and f /= 0, the v = $(-1)^s 2^{-1022} (0.f)$
    - De-normalized numbers – allow for graceful underflow
  - If e = 0 and f = 0 the v = $(-1)^s 0$ (zero)

IEEE Floating-Point Format

	single: 8 bits double: 11 bits	single: 23 bits double: 52 bits

S	Exponent	Fraction

- S: sign bit (0 ⇒ non-negative, 1 ⇒ negative)
- Normalize significand: 1.0 ≤ |significand| < 2.0
  - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
  - Significand is Fraction with the “1.” restored
- Exponent: excess representation: actual exponent + Bias
  - Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1203

Single-Precision Range

LNMIIT
The LNM Institute of Information Technology

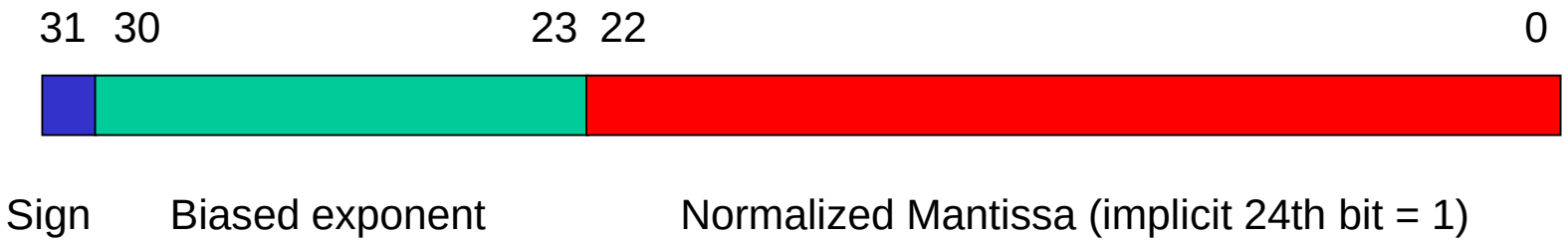
- Exponents 00000000 and 11111111 reserved
- Smallest value
  - Exponent: 00000001
    $\Rightarrow$ actual exponent = 1 – 127 = –126
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
  - exponent: 11111110
    $\Rightarrow$ actual exponent = 254 – 127 = +127
  - Fraction: 111…11 $\Rightarrow$ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Double-Precision Range

- Exponents 0000…00 and 1111…11 reserved
- Smallest value
  - Exponent: 00000000001
    $\Rightarrow$ actual exponent = 1 – 1023 = –1022
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value
  - Exponent: 11111111110
    $\Rightarrow$ actual exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 $\Rightarrow$ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Representation of Floating Point Numbers

- **IEEE 754 single precision**



$$(-1)^s \times F \times 2^{E-127}$$

Exponent	Mantissa	Object Representec
0	0	0
0	non-zero	denormalized
1-254	anything	FP number
255	0	pm infinity
255	non-zero	NaN

Normalization

- FP numbers are usually normalized
- i.e. exponent is adjusted so that leading bit (MSB) of mantissa is 1
- Since it is always 1 there is no need to store it
- (c.f. Scientific notation where numbers are normalized to give a single digit before the decimal point
- e.g. 3.123×10^3)

FP Ranges

	Denormalized	Normalized	Approximate Decimal
Single Precision	$\pm 2^{-149}$ to $(1 - 2^{-23}) \times 2^{-126}$	$\pm 2^{-126}$ to $(2 - 2^{-23}) \times 2^{127}$	± approximately $10^{-44.85}$ to approximately $10^{38.53}$
Double Precision	$\pm 2^{-1074}$ to $(1 - 2^{-52}) \times 2^{-1022}$	$\pm 2^{-1022}$ to $(2 - 2^{-52}) \times 2^{1023}$	± approximately $10^{-323.3}$ to approximately $10^{308.3}$

Floating-Point Number Format and Distribution

LNMIIT
The LNM Institute of Information Technology

Floating-point number format.

±	e	s

Sign

0 : +
1 : –

Exponent:
Signed integer,
often represented
as unsigned value
by adding a bias

Range with h bits:
$[-\text{bias}, 2^h-1-\text{bias}]$

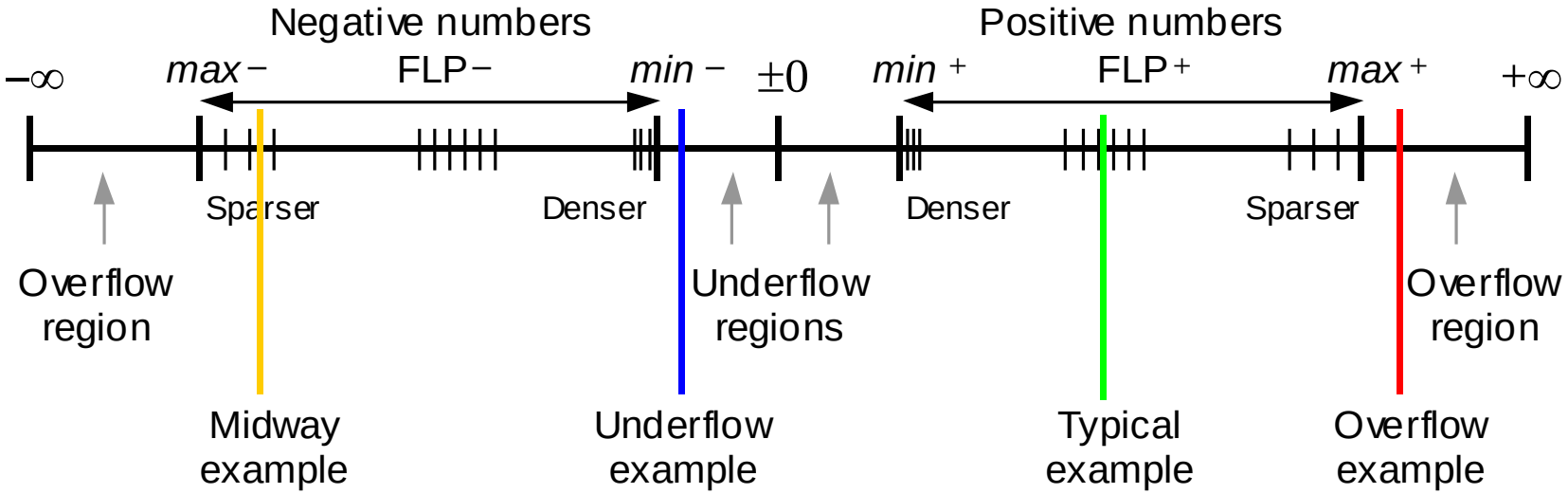
Significand:
Represented as a fixed-point number

Usually normalized by shifting,
so that the MSB becomes nonzero.
In radix 2, the fixed leading 1
can be removed to save one bit;
this bit is known as "hidden 1".

1.001×2^{-5}

$1.001 \times 2^{+7}$

Subranges and special values in floating-point number representations.



Floating-Point Example

- Represent –0.75
  - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
  - S = 1
  - Fraction = $1000\ldots00_2$
  - Exponent = –1 + Bias
    - Single: –1 + 127 = 126 = 01111110_2
    - Double: –1 + 1023 = 1022 = 01111111110_2
- Single: 1011 1111 0100 0…00
  - In hex representation : BF40 0000
- Double: 1011 1111 1110 1000…00
  - In hex representation : 2FFA 0000 0000 0000 0000

Floating-Point Example

- Represent 100
  - $100 = (-1)^0 \times 1.1001_2 \times 2^6$
  - S = 0
  - Fraction = $10010\ldots00_2$
  - Exponent = –1 + Bias
    - Single: 6 + 127 = 133 = $1000\ 0101_2$
- Single: 0100 0010 1100 1000 0000 0000 0000 ….
  - In hex representation : 42C8 0000

Floating-Point Example

- What number is represented by the single-precision float

  110000001 01000…00

  – S = 1

  – Fraction = $01000\ldots00_2$

  – Fxponent = $10000001_2 = 129$

- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$

  $= (-1) \times 1.25 \times 2^2$

  $= -5.0$

Another example

- Representation for -175
  - 175 = 128 + 32 + 8 + 4 + 2 +1 = 1010 1111
  - Or 1.0101111×2^7
  - S = 1
  - Exponent is 7 +127 = 134 = 1000 0110
  - Fractional part f = 0101111
  - Representation 1100 0011 0010 1111 0000 ….
  - Or in Hex $C32F 0000

Converting back

- Convert $C32F 0000 into decimal
- Extract components from
  - 1100 0011 0010 1111
  - S = 1
  - Exponent = 1000 0110 = 128+4+2 = 134
  - unbias 134 – 127 =7
  - f = 0101111 so mantissa is 1.0101111
  - Adjust by exponent 1010 1111 (move binary pt 7 places)
  - Or 128+32+15 = 175
  - Sign is negative so -175

Another example

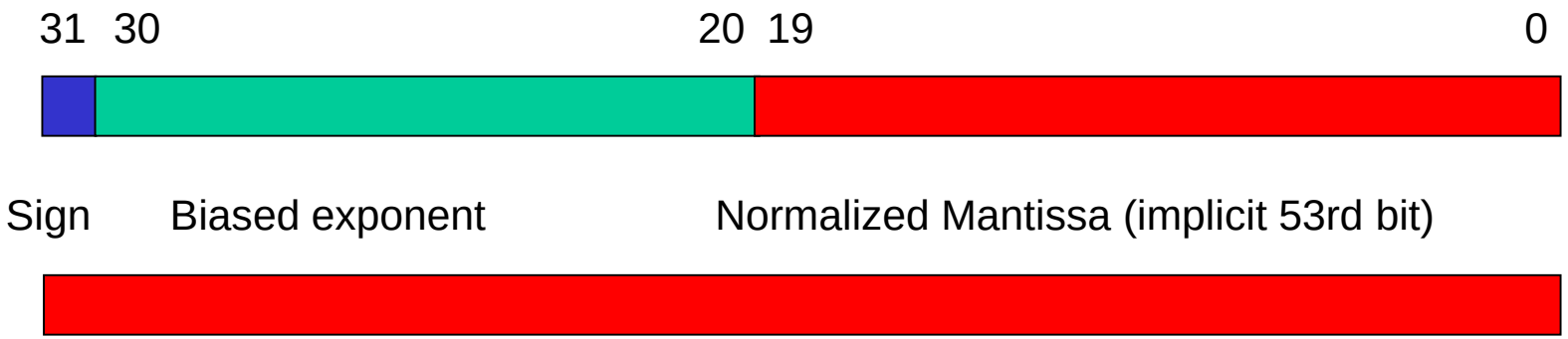
- Convert $41C8 0000 to decimal
  - 0100 0001 1100 1000 0000 ….
  - S is 0 so positive number
  - Exponent 1000 0011 = 128+3=131-127=4
  - f = 1001 so mantissa is 1.1001
  - With 4 binary positions have 11001 as final number or a decimal
  - 25

Overflow and Underflow

- **Overflow**
  - Result is too large (> 2.0×10^{38})
  - Exponent larger than represented in 8-bit Exponent field
- **Underflow**
  - Result is too small
    - >0, < 2.0×10^{-38}
  - Negative exponent larger than represented in 8-bit Exponent field

Representation of Floating Point Numbers

- **IEEE 754 double precision**



$$(-1)^s \times F \times 2^{E-1023}$$

Exponent	Mantissa	Object Represented
0	0	0
0	non-zero	denormalized
1-2046	anything	FP number
2047	0	pm infinity
2047	non-zero	NaN

Overview of IEEE 754-2008 Standard Formats

Feature	Single / Short	Double / Long
Word width (bits)	32	64
Significand bits	23 + 1 hidden	52 + 1 hidden
Significand range	$[1, 2 - 2^{-23}]$	$[1, 2 - 2^{-52}]$
Exponent bits	8	11
Exponent bias	127	1023
Zero (± 0)	$e + bias = 0$, $f = 0$	$e + bias = 0$, $f = 0$
Denormal	$e + bias = 0$, $f \neq 0$	$e + bias = 0$, $f \neq 0$
	represents $\pm 0.f \times 2^{-126}$	represents $\pm 0.f \times 2^{-1022}$
Infinity ($\pm\infty$)	$e + bias = 255$, $f = 0$	$e + bias = 2047$, $f = 0$
Not-a-number (NaN)	$e + bias = 255$, $f \neq 0$	$e + bias = 2047$, $f \neq 0$
Ordinary number	$e + bias \in [1, 254]$	$e + bias \in [1, 2046]$
	$e \in [-126, 127]$	$e \in [-1022, 1023]$
	represents $1.f \times 2^e$	represents $1.f \times 2^e$
min	$2^{-126} \cong 1.2 \times 10^{-38}$	$2^{-1022} \cong 2.2 \times 10^{-308}$
max	$\cong 2^{128} \cong 3.4 \times 10^{38}$	$\cong 2^{1024} \cong 1.8 \times 10^{308}$

FP Arithmetic +/-

- Check for zeros
- Align significands (adjusting exponents)
- Add or subtract significands
- Normalize result

Floating-Point Addition

Consider a 4-digit decimal example

$9.999 \times 10^1 + 1.610 \times 10^{-1}$

1. Align decimal points
   - Shift number with smaller exponent
   - $9.999 \times 10^1 + 0.016 \times 10^1$
2. Add significands
   - $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$
3. Normalize result & check for over/underflow
   - 1.0015×10^2
4. Round and renormalize if necessary
   - 1.002×10^2

Floating-Point Addition

Now consider a 4-digit binary example

- $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + –0.4375)

1. Align binary points
   - Shift number with smaller exponent
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
2. Add significands
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-}1 = 0.001_2 \times 2^{-1}$
3. Normalize result & check for over/underflow
   - $1.000_2 \times 2^{-4}$, with no over/underflow
4. Round and renormalize if necessary
   - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

FP Arithmetic x/÷

LNMIIT
The LNM Institute of Information Technology

- Check for zero
- Add/subtract exponents
- Multiply/divide significands (watch sign)
- Normalize
- Round
- All intermediate results should be in double length storage

Floating-Point Adders/Subtractors

LNMIIT
The LNM Institute of Information Technology

Floating-Point Addition Algorithm

Assume $e1 \geq e2$; *alignment shift* (*preshift*) is needed if $e1 > e2$

$$(\pm s1 \times b^{e1}) + (\pm s2 \times b^{e2}) = (\pm s1 \times b^{e1}) + (\pm s2 / b^{e1-e2}) \times b^{e1}$$
$$= (\pm s1 \pm s2 / b^{e1-e2}) \times b^{e1} = \pm s \times b^{e}$$

Example:

Numbers to be added:

$x = 2^5 \times 1.00101101$

$y = 2^1 \times 1.11101101$ ← Operand with smaller exponent to be preshifted

Operands after alignment shift:

$x = 2^5 \times 1.00101101$

$y = 2^5 \times 0.000111101101$

Result of addition:

$s = 2^5 \times 1.010010111101$ ← Extra bits to be rounded off

$s = 2^5 \times 1.01001100$ ← Rounded sum

Like signs:
Possible 1-position normalizing right shift

Different signs:
Left shift, possibly by many positions

Overflow/underflow
during addition or normalization

Floating Point Addition Hardware

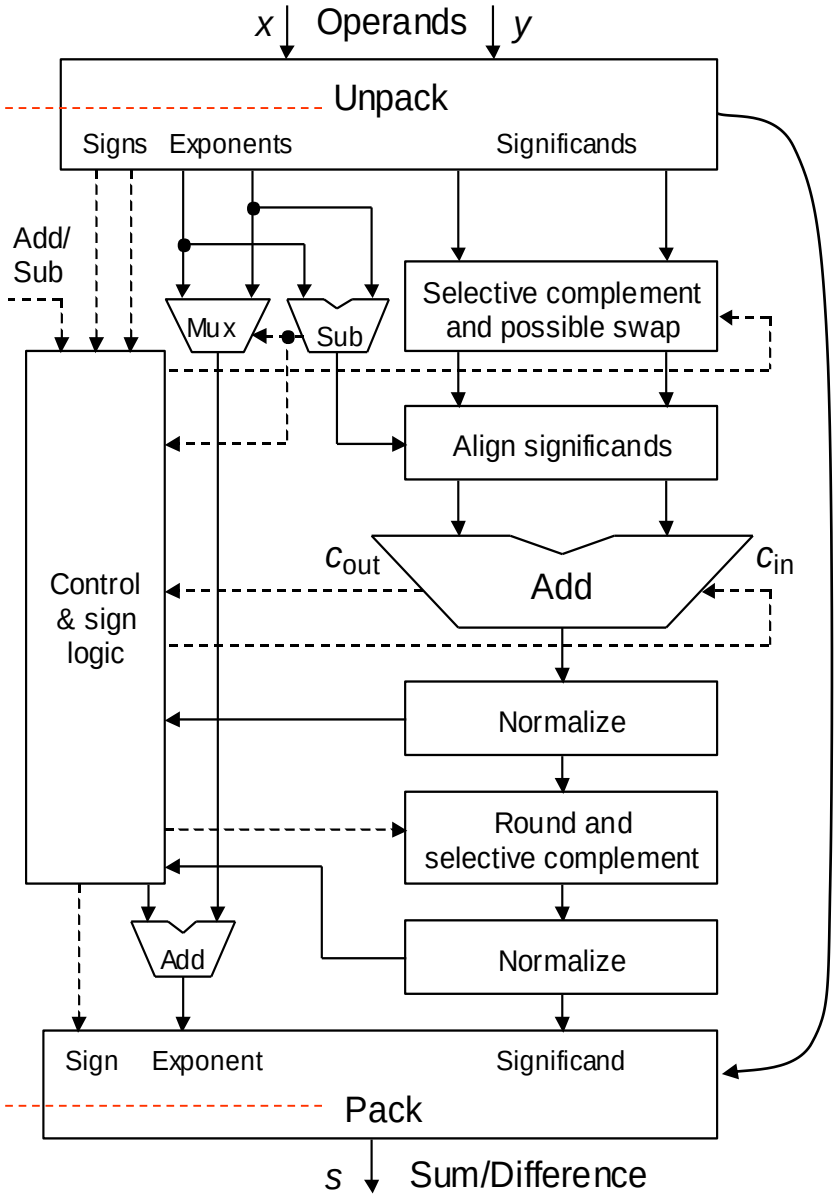
Isolate the sign, exponent, significand
Reinstate the hidden 1
Convert operands to internal format
Identify special operands, exceptions

Block diagram of a floating-point adder/subtractor.

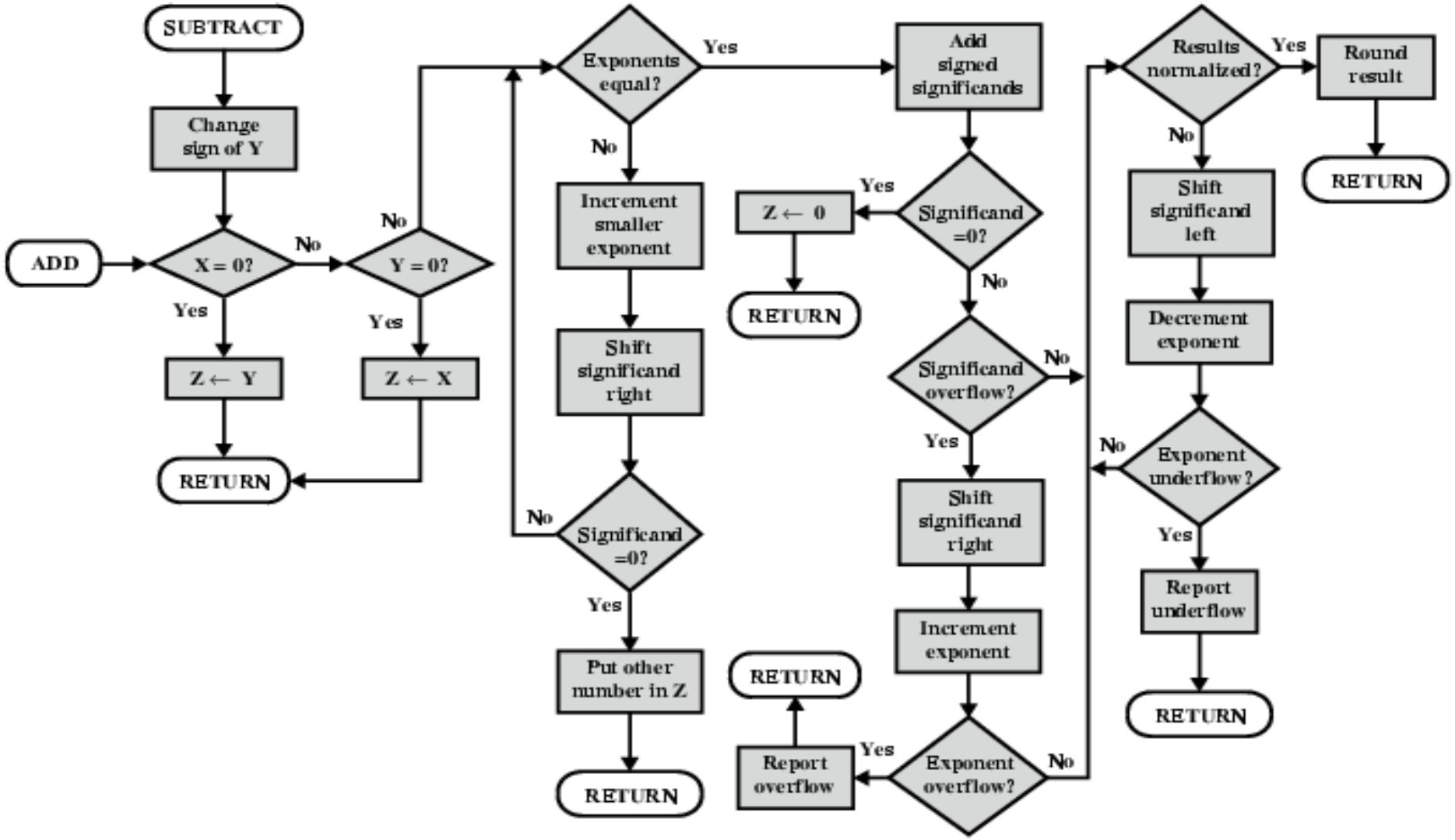
Other key parts of the adder:
Significand aligner (preshifter)
Result normalizer (postshifter), including leading 0s detector/predictor
Rounding unit
Sign logic

Converting internal to external representation, if required, must be done at the rounding stage

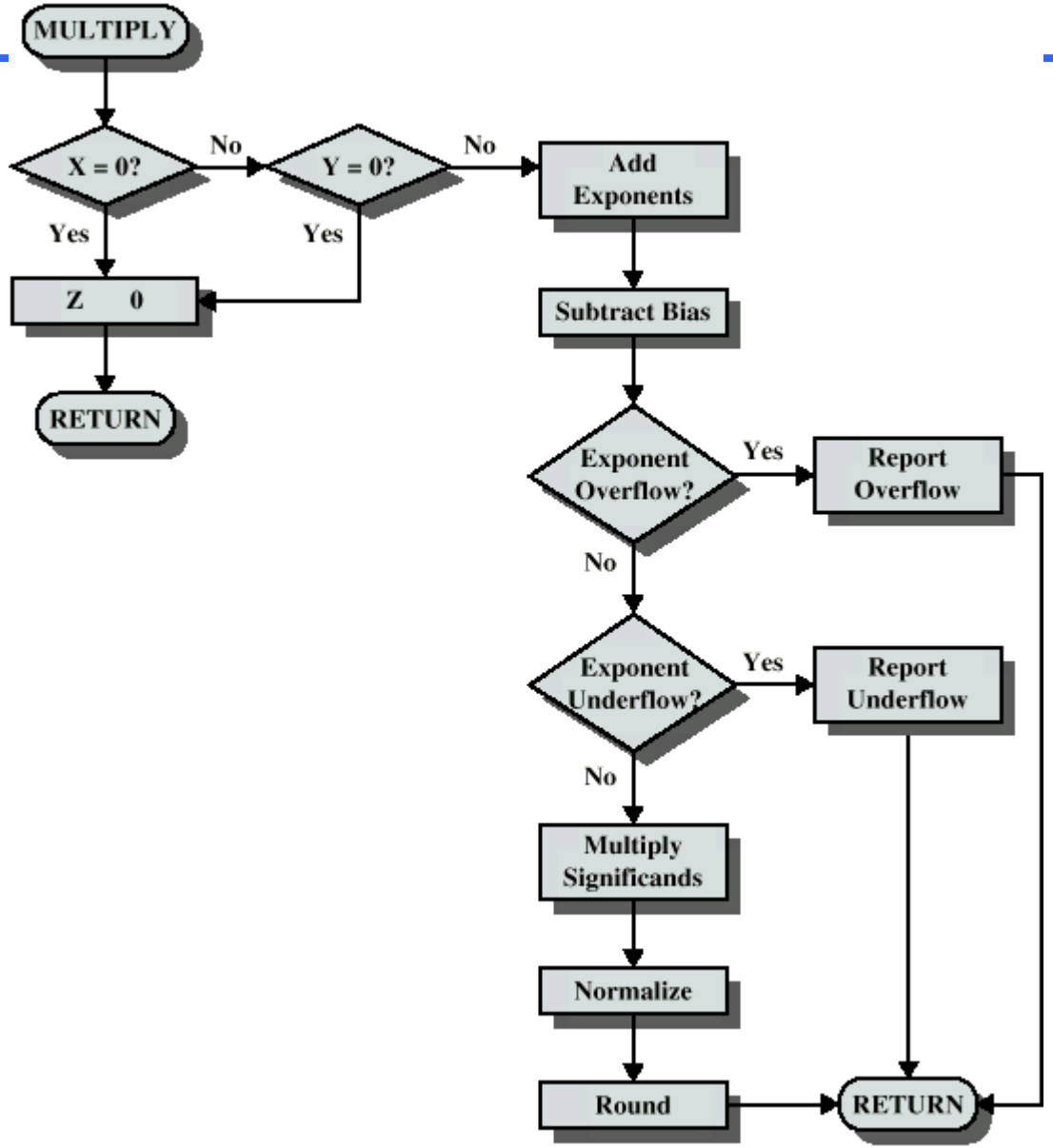
Combine sign, exponent, significand
Hide (remove) the leading 1
Identify special outcomes, exceptions



FP Addition & Subtraction Flowchart



Floating Point Multiplication



Floating-Point Multipliers

$$(\pm\, s1 \times b^{e1}) \times (\pm\, s2 \times b^{e2}) \;=\; (\pm\, s1 \times s2) \times b^{e1+e2}$$

$s1 \times s2 \in [1, 4)$: may need postshifting

Overflow or underflow can occur during multiplication or normalization

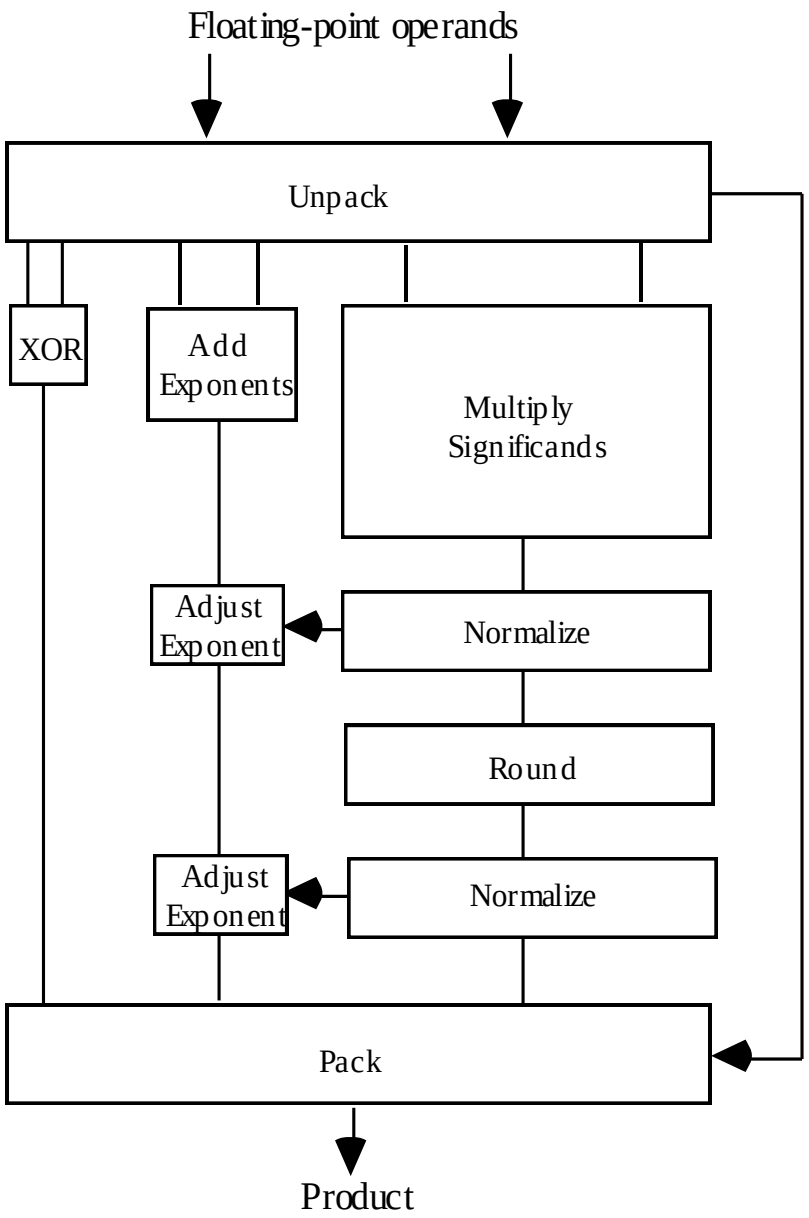
Speed considerations

Many multipliers produce the lower half of the product (rounding info) early

Need for normalizing right-shift is known at or near the end

Hence, rounding can be integrated in the generation of the upper half, by producing two versions of these bits

Block diagram of a floating-point multiplier



Floating-Point Dividers

$$(\pm s1 \times b^{e1}) / (\pm s2 \times b^{e2}) = (\pm s1/s2) \times b^{e1-e2}$$

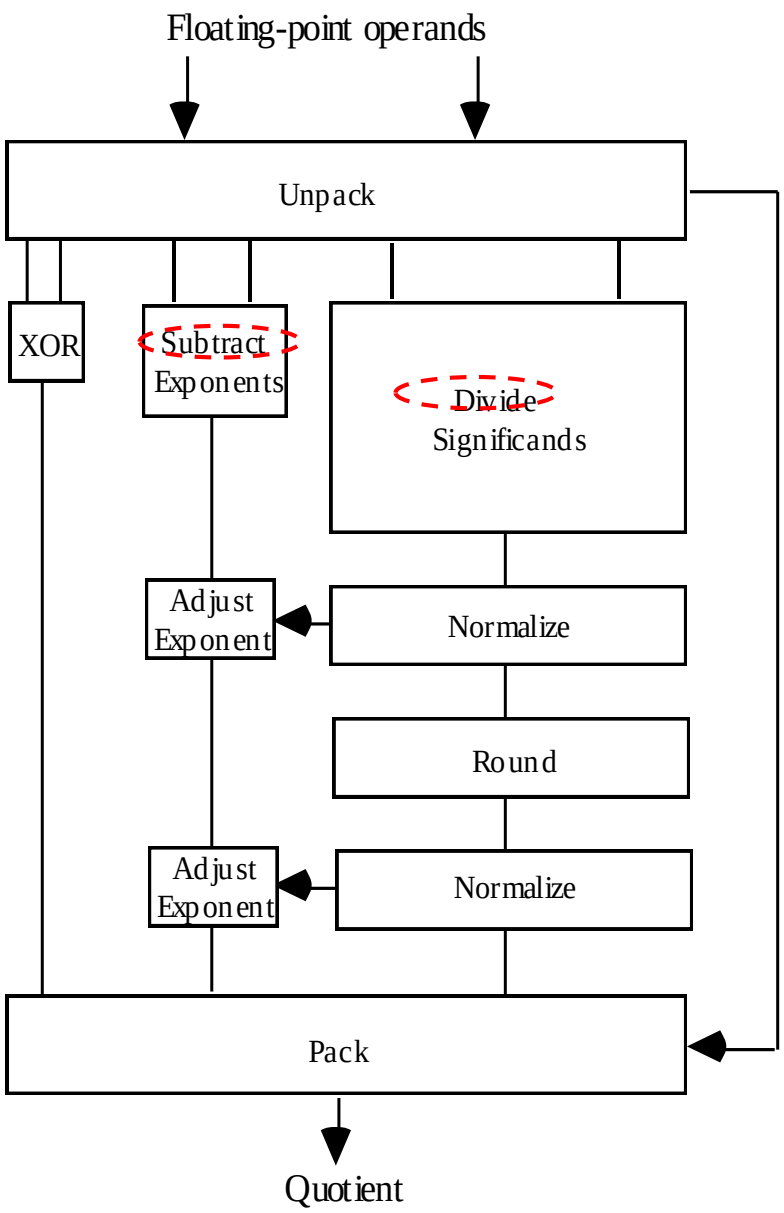
$s1/s2 \in (0.5, 2)$: may need postshifting

Overflow or underflow can occur during division or normalization

Note: Square-rooting never leads to overflow or underflow

Rounding considerations

Quotient must be produced with two extra bits (*G* and *R*), in case of the need for a normalizing left shift

The remainder acts as the sticky bit

Block diagram of a floating-point divider